

EduMetric CRM — Master Document

Version: 1.0 **Date:** May 2026 **Audience:** Hackathon jury, team, future investors and contributors **Status:** Pre-development
blueprint, locked scope

Table of Contents

1. [Product Vision](#)
 2. [Product Philosophy](#)
 3. [Project Goals](#)
 4. [User Roles](#)
 5. [Navigation Structure](#)
 6. [Product Sitemap](#)
 7. [Global Layout Structure](#)
 8. [Login Page](#)
 9. [Student Dashboard](#)
 10. [Teacher Dashboard](#)
 11. [Admin Dashboard](#)
 12. [Attendance UX](#)
 13. [Student Analytics](#)
 14. [Charts & Visualizations](#)
 15. [UI Component System](#)
 16. [Interaction Design](#)
 17. [Responsive Design](#)
 18. [Backend Architecture](#)
 19. [Frontend Architecture](#)
 20. [Database Structure](#)
 21. [API Structure](#)
 22. [Metrics Engine](#)
 23. [Scoring Formula](#)
 24. [Backend Modules](#)
 25. [Frontend Pages](#)
 26. [Development Workflow](#)
 27. [Git Workflow](#)
 28. [DevOps Structure](#)
 29. [Demo Strategy](#)
 30. [WOW-Effect Elements](#)
 31. [Biggest Risks](#)
 32. [Final Product Vision](#)
-

1. Product Vision

EduMetric CRM is a multi-dimensional student evaluation system that measures student growth across grades, attendance, practical work, behavior, activity, and consistency — not just GPA.

A single number cannot describe a person. Traditional educational systems still rely on GPA as the primary measure of student development, but employers, educators, and students themselves know GPA is a thin slice of reality. EduMetric replaces the single number with a **multi-dimensional profile** and shows real **growth dynamics** over time.

Target users: IT academies, university programs, PDP-style organizations, private education centers — primarily in the CIS / Central Asia region, with global potential.

Core promise: *Student growth analytics, not just grades.*

2. Product Philosophy

EduMetric is built on four principles. Every product decision must align with these.

2.1 Transparency over magic

The composite score is a transparent formula, not a black-box AI. Any user can click the score and see the formula, the weights, and how the number was computed. Administrators can adjust weights. This is a deliberate ethical stance: when decisions affect student outcomes, opacity is not acceptable.

2.2 Growth over snapshots

Static numbers tell you nothing. EduMetric is built around **change over time** — every dashboard prioritizes trends, deltas, and trajectory. A student rising from 50 to 70 is doing better than a student stuck at 75.

2.3 Speed over completeness

Teachers will not use a system that wastes their time. Attendance input must take seconds. If teacher input UX fails, all downstream analytics are garbage. UX velocity is treated as a first-class technical concern.

2.4 Humanize the metric

A radar chart is not a person. Every screen reminds the user that a number represents a human being: by adding names, photos, comments, and qualitative context wherever a number alone would feel cold.

3. Project Goals

3.1 Hackathon goals (immediate, 5 days)

- A working end-to-end MVP demonstrating the core insight: multi-dimensional, growth-aware student evaluation.
- A stable demo that runs reliably during jury presentation.
- A clear, professional product narrative that stands up to non-technical and technical scrutiny.

3.2 Product goals (post-hackathon)

- A real product usable by an actual educational institution.
- A foundation that can scale technically and organizationally.
- A defensible position in the EdTech analytics space, differentiated by transparency and growth-first approach.

3.3 Non-goals (deliberately out of scope)

- AI/ML predictions of student outcomes
 - Mobile native applications
 - Multi-tenant SaaS billing
 - LMS replacement (course delivery, content management)
 - Chat, video, messaging features
 - Gamification (badges, leaderboards)
 - Third-party integrations (Moodle, Canvas, etc.)
-

4. User Roles

The system supports three roles: **Admin**, **Teacher**, **Student**. No further granularity in MVP.

4.1 Admin

Profile: director of school, program coordinator, system administrator.

Sees:

- High-level KPI across the organization
- Full lists of students, teachers, groups, courses
- Analytics dashboards and group comparisons
- Settings: formula weights, user management, optional audit log

Can do:

- Create / edit / soft-delete students, teachers, groups, courses
- Assign students to groups, teachers to courses
- Adjust scoring formula weights (with confirmation, triggers recompute)
- View at-risk lists across the entire organization
- View teacher activity (which teachers are actually inputting data)

Cannot do:

- Input attendance for classes (this is a teacher action)
- Read private teacher comments on students (privacy boundary)

4.2 Teacher

Profile: instructor running one or more groups and courses.

Sees:

- Today's lessons schedule
- Groups they teach with aggregate metrics
- Students in those groups with individual metrics
- At-risk students within their groups, with reasons
- Pending tasks: unmarked attendance, ungraded assignments

Can do (quick actions, one-click priority):

- Mark attendance for a lesson (bulk, default = all present)
- Grade an assignment (single or bulk)
- Add behavior or activity record for a student (periodic, 1–5 scale)
- Open any student profile in their groups

Cannot do:

- Modify other teachers' grades or comments
- See students outside their assigned groups
- Change scoring formula

4.3 Student

Profile: the most emotionally important user — drives demo impact.

Sees:

- Personal dashboard: composite score, radar profile, trend
- Detailed profile with deeper analytics
- All personal grades and attendance
- Comparison with group average (anonymized — "you are in top X%")
- Growth areas with suggestions

Cannot do:

- Edit anything in their own metrics
- See other students by name
- See a numeric rank like "#5 of 18" (privacy + ethics) — only percentile ranges
- Read teacher behavior comments (only numeric values)

5. Navigation Structure

All roles share the global layout (sidebar + topbar). Sidebar content varies by role.

5.1 Student Sidebar

EduMetric (logo)

- △ Dashboard → main screen: radar, trend, score, growth areas
- My Profile → full profile with bio and detail metrics
- My Growth → deep dive into growth dynamics, per-dimension history
- Attendance → all my attendance with heatmap
- Grades → all my grades, filterable by course

⚙ Settings

↪ Log out

5.2 Teacher Sidebar

EduMetric

- △ Today → today's lessons, quick actions
- My Groups → grid of groups with aggregate metrics
- Attendance → attendance management entry point
- Grades → grading queue + history
- ⚠ At Risk → students requiring attention in my groups
- Students → all my students, searchable

⚙ Settings

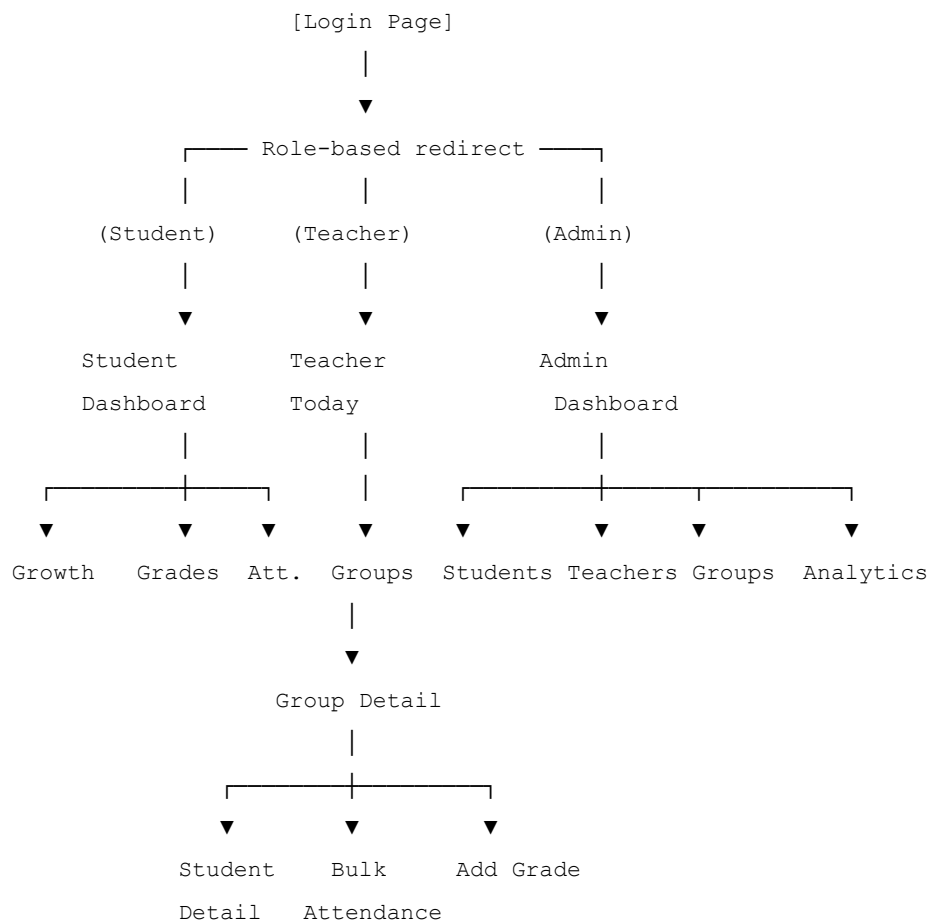
↪ Log out

5.3 Admin Sidebar

EduMetric

- △ Dashboard → organization-level KPI
- Students
- Teachers
- Groups
- Courses
- Analytics → deep analytics, comparisons
- △ At Risk
-
- ⚙ Settings
 - | Formula Config
 - | Users & Roles
 - | Audit Log (optional)
- ↪ Log out

6. Product Sitemap



6.1 Critical flows

Flow A — Teacher takes attendance (most frequent): Today → click lesson → BulkAttendanceForm → toggle exceptions → Save Target: 4 clicks max, under 30 seconds for a group of 20.

Flow B — Student checks progress: Login → Dashboard (auto) → scroll → optional click into Growth Areas or radar dimension drill-down

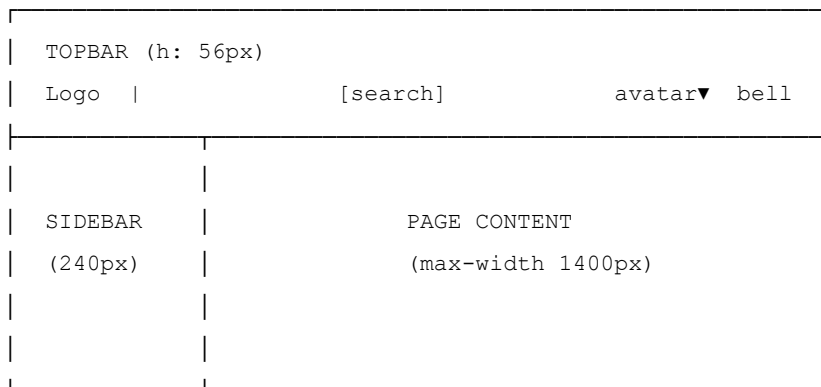
Flow C — Admin handles at-risk: Dashboard → "X at risk" card → At Risk list → click student → Student Profile → action

Flow D — Admin adjusts formula: Settings → Formula Config → adjust sliders → preview → Apply (with warning modal)

6.2 Page build priority

- **P0 (must work):** Login, Student Dashboard, Teacher Today, Bulk Attendance, Student Profile, Admin Dashboard
- **P1 (should work):** My Groups, Group Detail, Grades input, At Risk list
- **P2 (nice):** My Growth deep dive, Formula Config, Audit Log

7. Global Layout Structure



- **Topbar:** logo, optional search, notification bell, user avatar with dropdown (profile, settings, logout).
- **Sidebar:** role-based navigation. Collapses to 64px (icons only) on screens < 1024px. Hidden behind hamburger on screens < 768px.
- **Page content area:** max-width 1400px, centered, padding px-6 py-8. All pages share this container for visual consistency.

7.1 Visual identity

- **Style:** modern minimal SaaS analytics. References: Linear, Vercel, Notion, Stripe Dashboard.
- **Mode:** Light mode first. Dark mode optional (built only if time allows).
- **Color palette:**
 - Background: zinc-50, cards white
 - Border: zinc-200

- **Text:** zinc-900 / zinc-500 / zinc-400
- **Accent:** indigo-600 (single accent across the product)
- **Semantic (sparing use):** emerald-500 **success**, amber-500 **warning**, rose-500 **danger**, sky-500 **info**
- **Typography:** Inter only. Display sizes for hero numbers (text-5xl/6xl), text-sm (14px) as body default, tabular-nums for all numeric data.
- **Spacing:** standard rhythm 4/8/16/24/32/48px. Card padding p-6, gap between cards gap-6.
- **Borders & radius:** 1px borders only, never 2px. Radius: rounded-lg (8px) cards, rounded-md (6px) inputs/buttons, rounded-full avatars/badges.
- **Shadows:** very subtle. shadow-sm default, shadow-md on hover, shadow-xl on modals.
- **No glassmorphism.** Only place backdrop blur appears: behind modals.
- **Animations:** transition-all duration-150 ease-out standard. Recharts default animations at 400ms. Skeleton on load, not spinners. Count-up animations for hero numbers.
- **Icons:** Lucide (bundled with shadcn). 16/20px. Stroke 1.5.

8. Login Page

8.1 Layout

Split screen, 60/40 desktop. Fullscreen card on mobile.

[Hero / brand side]	[Login form card]
- subtle gradient bg	
- tagline	Logo
- mini animated radar	"Welcome back"
"Student growth analytics, not just grades."	[Email input]
	[Password input]
	[Sign in →]
	Forgot password?

8.2 Hero side

- **Background:** subtle gradient from-indigo-50 via-white to-zinc-50
- **Big tagline** (text-4xl font-semibold)
- Small **animated mini radar chart**, slow rotation/pulse — immediately telegraphs "analytics product"
- Logo top-left

8.3 Form side

- Centered card, max-width 400px
- Logo → heading → fields → button

- Inputs: shadow Input with label above
- Button: full-width, primary, "Sign in →"
- Loading state: button disabled, "Signing in...", spinner replaces arrow
- Error state: red banner above form, "Invalid email or password"

8.4 Demo credentials

Collapsible "Try demo accounts" link, expands to:

- **Admin:** admin@edumetric.io / Demo2026!
- **Teacher:** teacher@edumetric.io / Demo2026!
- **Student:** student@edumetric.io / Demo2026!

Optionally three "Sign in as ..." shortcut buttons for jury convenience.

8.5 Interactions

- Email autofocus on load
- Tab order: email → password → button
- Enter in password submits
- Smooth fade transition to dashboard on success

9. Student Dashboard

The single most important screen in the product. The demo opens here 80% of the time.

9.1 Layout

HERO			
Avatar	Name + meta	[78.4]	↑+5.2
		composite growth	
		[Top 25% in group]	

RADAR		TREND	
Your Profile		Your Growth	
5-dim radar chart		12-week line chart	
student vs group		smooth + gradient	

BREAKDOWN			
Grades 82	Attendance 91	Practical 71	...
row per dimension with progress bar + delta vs group			

GROWTH AREAS		RECENT ACTIVITY	
• Practical (-9.5)		Tab: Grades · Att.	
• Activity (-13)		Last 5 entries	

9.2 Hero section

- **Left:** avatar (64px), full name `text-2xl font-semibold`, meta line `text-sm text-zinc-500: "Group · Semester · Enrolled date"`
- **Right:** composite score `text-6xl font-semibold tabular-nums` in `indigo-600`; label below; growth arrow `↑ +5.2` colored by direction; badge "Top 25% in group" below.

Interactions:

- Hover composite → tooltip "Based on 47 data points · Updated today"
- **Click composite** → **transparency modal** with full formula breakdown (key wow-effect, see section 30)
- Hover growth arrow → tooltip "Compared to previous 4-week period"

9.3 Radar chart card

- Title "Your Profile", subtitle "5 dimensions of your growth"
- 5 axes: Grades, Attendance, Practical, Behavior, Activity (scale 0–100)
- Two overlays: student (filled `indigo-500/40`, stroke `indigo-600`) + group avg (dashed `zinc-400`, no fill)
- Animation: polygon expands from center on load (600ms ease-out)
- Click axis label → navigates to My Growth focused on that dimension

9.4 Trend chart card

- Title "Your Growth", subtitle "Last 12 weeks"
- LineChart: weeks X, composite Y, smooth monotone curve, indigo stroke, gradient fill underneath
- Dots at each point, **glowing pulsing dot on the last point** (current state)
- Hover: vertical reference line + tooltip "Week 8: 73.2 · ↑ +2.1"
- Click point → modal "What changed that week?"

9.5 Breakdown card

Full-width. Five rows (one per dimension):

```
[icon] Grades      82.0 / 100
      Group avg: 78.5  [■■■■■■■■■■] 82
      ↑ +3.5 vs group
```

Row hover → `bg-zinc-50`. Click → inline accordion expand with 8-week sparkline + sub-metrics.

9.6 Growth areas card

- Title "Where to grow"
- 2–3 rows of weakest dimensions: name, value, delta in red, static suggestion ("Engage more in lab work")
- Subtle `bg-amber-50/50` background to differentiate as an alert area

9.7 Recent activity card

- Tabs: Grades · Attendance
- Last 5 items with subject icon, name, value, date
- "View all →" footer link

10. Teacher Dashboard

Optimized for maximum-velocity input. The teacher home screen is **Today**.

10.1 Layout

Heading: "Today, Friday May 15"

Sub: "3 lessons · 2 pending grades · 5 at-risk students"

Today's Lessons (horizontal cards)		
09:00 JS	11:00 Algo	14:00 Workshop
Backend-24	Frontend-24	Backend-24
☐ 22/24 ✓	[Mark Att.]	[Mark Att.]

Pending Tasks	At-Risk in Your Groups
• Grade Lab #3 (24)	⚠ Anvar K. (-15% trend)
• Grade Lab #4 (18)	⚠ Mavluda S. (low att.)
• Review behavior week	→ View all 5

My Groups Overview				
Backend-2024	18 students	avg 73.4	↑ +1.2	
Frontend-2024	22 students	avg 68.9	↓ -0.5	
JS-Intensive	14 students	avg 81.2	↑ +3.1	

10.2 Today's lessons cards

Each card states time, course, group, plus a status indicator:

- Past + attendance done: ✓ 22/24 attended
- Past + attendance pending: red badge "Mark now" + CTA
- Upcoming: countdown "Starts in 2h"
- Current: pulsing green dot "Live now" + prominent CTA

Click → Bulk Attendance form for that lesson.

10.3 Pending tasks

Linkable list of actionable items (grading queue, behavior records due).

10.4 At-risk sidebar

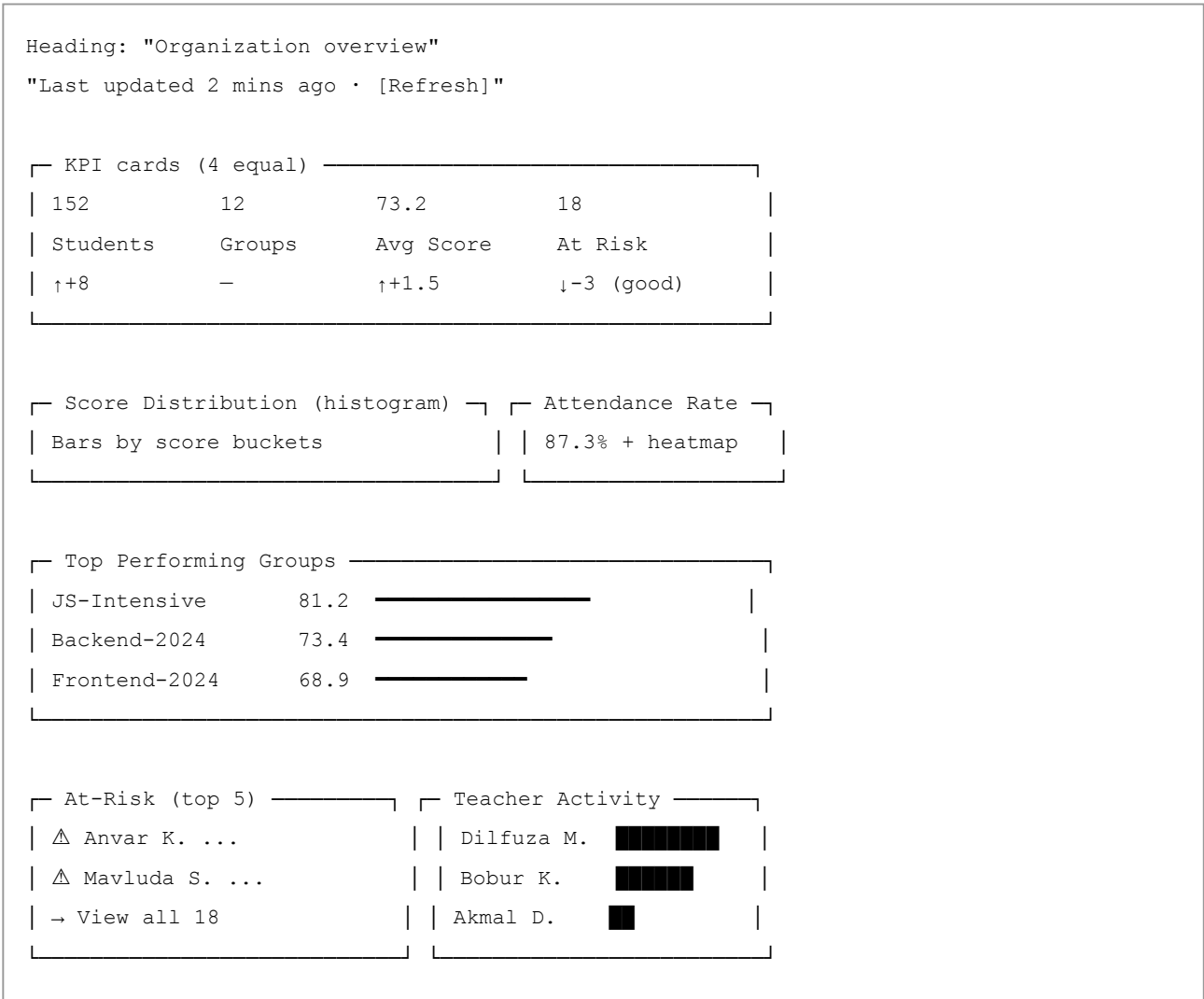
3–4 students max with concise reason. Overflow to "View all → X more". Click → student profile.

10.5 My Groups overview

Sleek table without grid lines: name, student count, avg score, change indicator. Sortable. Click → group detail.

11. Admin Dashboard

11.1 Layout



11.2 KPI cards

Big number text-4xl font-semibold tabular-nums, label below, trend sub-line. Click "At Risk" → at-risk page. Click "Students" → list.

11.3 Score distribution histogram

Bars per score bucket (0–20, 20–40, 40–60, 60–80, 80–100). Single accent color. Hover tooltip with student count.

11.4 Top groups bar chart

Horizontal bars ranked by avg score. Hover → link to group.

11.5 At-risk panel

Top 5 highest-risk students with critical reason. Click → student profile.

11.6 Teacher activity panel

Unique meta-metric: which teachers are actually inputting data. Horizontal bars of data points entered in last 7 days. Low activity highlighted in amber.

12. Attendance UX

This is the most frequently used flow in the system. If it is slow, the product dies.

12.1 Layout

← Back to Today

Mark attendance

Algorithms · Frontend-2024 · Friday May 15, 11:00

[Mark all present]

[Mark all absent]

[Reset]

#	Name	Present	Late	Absent	Excused	Note
1	Alibek Karimov	☒	☐	☐	☐	
2	Anvar Komilov	☐	☐	☒	☐	
3	Bek Yusupov	☒	☐	☐	☐	
4	Dilshod Ahmedov	☐	☒	☐	☐	
5	Mavluda Sattorova	☐	☐	☐	☒	

- `↓ ↑` — move focus row by row
- `1 2 3 4` — set status of focused row
- `N` — open note input
- `Cmd+S / Ctrl+S` — save
- `Esc` — close with unsaved warning

Focused row highlighted with `bg-zinc-50 + left border border-l-2 border-indigo-500`.

12.4 Save flow

- Click Save → loading state → toast "Attendance saved · 24 students" → auto-redirect to Today after 800ms
- On error: red banner above form, data preserved

12.5 Edge cases

- Future lesson: save allowed with warning
- Re-edit: loads previous values, button becomes "Update attendance"
- Network error: retry button + toast

12.6 Tablet behavior (iPad use case for in-class teachers)

- Touch targets $\geq 44\text{px}$
- Status buttons stay visible
- Sticky save bar bottom

13. Student Analytics

The deep-dive page for a single student. Accessed via student profile.

13.1 Profile header

[Avatar L]

Alibek Karimov

Backend-2024 · Semester 2 · Joined Sept 2024

alibek.k@edu.uz

[Composite: 78.4] [Rank: top 25%]

13.2 Tabs

Overview · Analytics · Grades · Attendance · Notes

13.3 Overview tab

- Larger radar + trend (as on dashboard)
- Performance breakdown table

- **Growth timeline** (vertical, Notion-style changelog):
 - "Apr 15 — Score jumped +4 points (better practical work)"
 - "Mar 28 — Attendance dropped to 70% (3 missed lessons)"
 - "Mar 10 — Top 30% milestone reached"

13.4 Analytics tab

For each of 5 dimensions:

- Current value, group avg, delta
- 12-week sparkline
- Sub-metric breakdown (e.g. Grades by assignment type)
- Optional attendance heatmap (52 weeks × days)

13.5 Grades tab

Sortable filterable table: assignment, date, type, value/max, grade %, averages footer.

13.6 Attendance tab

Big stat (91% rate), heatmap, recent absences list.

13.7 Notes tab (teacher/admin only — hidden from student)

List of behavior records: teacher, date, value, comment. Add new note button.

14. Charts & Visualizations

14.1 Principles

- One accent color (`indigo-600`) for primary data
- Gray (`zinc-300`) for comparisons / averages
- Semantic colors only when they carry meaning
- Tooltips dark on light mode, light on dark mode
- Animations 400–600ms ease-out
- Empty state: icon + "Not enough data yet"
- Loading: skeleton with pulse

14.2 Radar chart

Recharts `<RadarChart>`. 5–6 axes. Stroke `indigo-600`, fill `indigo-500` at opacity 0.4. Group avg overlay: dashed stroke, no fill. Polar grid `zinc-200`. Builds from center outward on load.

14.3 Line chart (trend)

Recharts `<LineChart>` with `<Line type="monotone">`. 2px indigo stroke. Gradient area underneath (indigo-600/20 → transparent). Dots on each point, glowing pulsing dot on the last. Reference line for group avg (horizontal dashed).

14.4 Bar chart

Vertical for distributions, horizontal for rankings. Single indigo color. Hover: full opacity. Optional bar-fill-from-bottom animation.

14.5 Heatmap (attendance)

Custom grid (Recharts doesn't do heatmaps well). Week columns × day cells. Color scale white → indigo-200 → indigo-500 → indigo-700. Hover tooltip with date and status. **Nice-to-have, not critical for MVP.**

14.6 Progress bars

Track `bg-zinc-100`, fill `bg-indigo-600`, height 6px, rounded full. Animated load (0 → value, 400ms).

14.7 Premium-feel tricks

- `tabular-nums` everywhere for numeric data
- Count-up animations for hero numbers
- Subtle gradients under line charts
- Glowing dot on last trend point
- Staggered chart entries on page load
- Custom empty-state illustrations (line-style, simple)

15. UI Component System

15.1 From shadcn/ui (no custom work)

`Button · Input · Textarea · Label · Card · Table · Dialog · DropdownMenu · Tabs · Badge · Toast (Sonner) ·`
`Tooltip · Avatar · Skeleton · Select · Switch · Checkbox · Sheet (slide-out drawer)`

15.2 Custom components (built on top)

- **ScoreBadge** — pill with number + color by value (red < 50, amber 50–70, green > 70)
- **GrowthIndicator** — ↑+5.2 or ↓-2.1 with color
- **MetricCard** — standard KPI card (number + label + trend)
- **DimensionRow** — row for breakdown (label + value + progress bar + comparison)
- **DataTable** — table with sorting/filtering/pagination wrapper
- **AtRiskBadge** — amber/red pill with ⚠ icon
- **EmptyState** — illustration + message
- **PageHeader** — title + subtitle + actions row
- **LoadingPage** — full skeleton for page load

- **AnimatedNumber** — count-up wrapper
- **StatusBadge** — for attendance statuses (Present/Late/Absent/Excused) with color

15.3 Layout components

- **AppShell** — Sidebar + Topbar + main area
 - **Sidebar** — collapsible navigation
 - **Topbar** — search, notifications, user menu
 - **PageContainer** — standard padding/max-width wrapper
-

16. Interaction Design

16.1 Hover states

- Cards: border `zinc-200` → `zinc-300`, optional `shadow-md lift`
- Buttons primary: `indigo-600` → `indigo-700`
- Buttons secondary: `zinc-50` → `zinc-100`
- Links: underline appears
- Table rows: `bg-zinc-50`
- Sidebar items: `bg-zinc-100`, **active** `bg-indigo-50 text-indigo-700`
- Icons: opacity 60% → 100%

All transitions: `transition-colors duration-150.`

16.2 Loading states

- **Page load**: full skeleton mirroring final layout
- **Inline action**: button shows "Saving...", spinner replaces icon, disabled
- **Data refresh**: subtle skeleton overlay over data area
- **Chart load**: skeleton shape (lines/bars in `zinc-200`)

No global spinner overlay.

16.3 Transitions

- Page navigation: Next.js default
- Modal open: fade + scale 0.95 → 1, 150ms
- Drawer slide-out: 200ms ease-out
- Toast: slide from top-right, auto-dismiss 4s

16.4 Empty states

Small line illustration (64px) + heading + helpful subtitle + optional CTA. Used wherever data may be absent.

16.5 Success / error states

- Success: toast for actions, inline checkmark for auto-saves
- Form errors: inline below field, red border on input
- Page errors (404/500): standalone page with illustration and "Go back" CTA
- API errors: toast (non-blocking) or modal (blocking)

16.6 Notifications

Topbar bell with count badge. Click → dropdown with recent items. Reality: placeholder for MVP, but visible UI element. 2–3 pre-seeded notifications for demo.

17. Responsive Design

17.1 Breakpoints (Tailwind defaults)

- `sm` 640px — phones landscape
- `md` 768px — tablets portrait
- `lg` 1024px — tablets landscape / small laptops
- `xl` 1280px — desktop default

17.2 Sidebar

- `> 1024px` — full sidebar 240px
- `768–1024px` — collapsed 64px (icons only)
- `< 768px` — hidden, hamburger opens it as Sheet drawer

17.3 Topbar

`< 640px` — search collapses behind icon; notifications and avatar remain.

17.4 Dashboard cards

- `> 1024px` — 2 columns for radar + trend
- `768–1024px` — 2 columns, but 3 KPI cards become 2+1
- `< 768px` — single column, stacked vertically

17.5 Charts

- Radar: shrinks proportionally
- Line: keeps responsive width, fewer X ticks
- Tables: horizontal scroll inside container, or card-row transform on very narrow screens

17.6 Attendance form on tablet

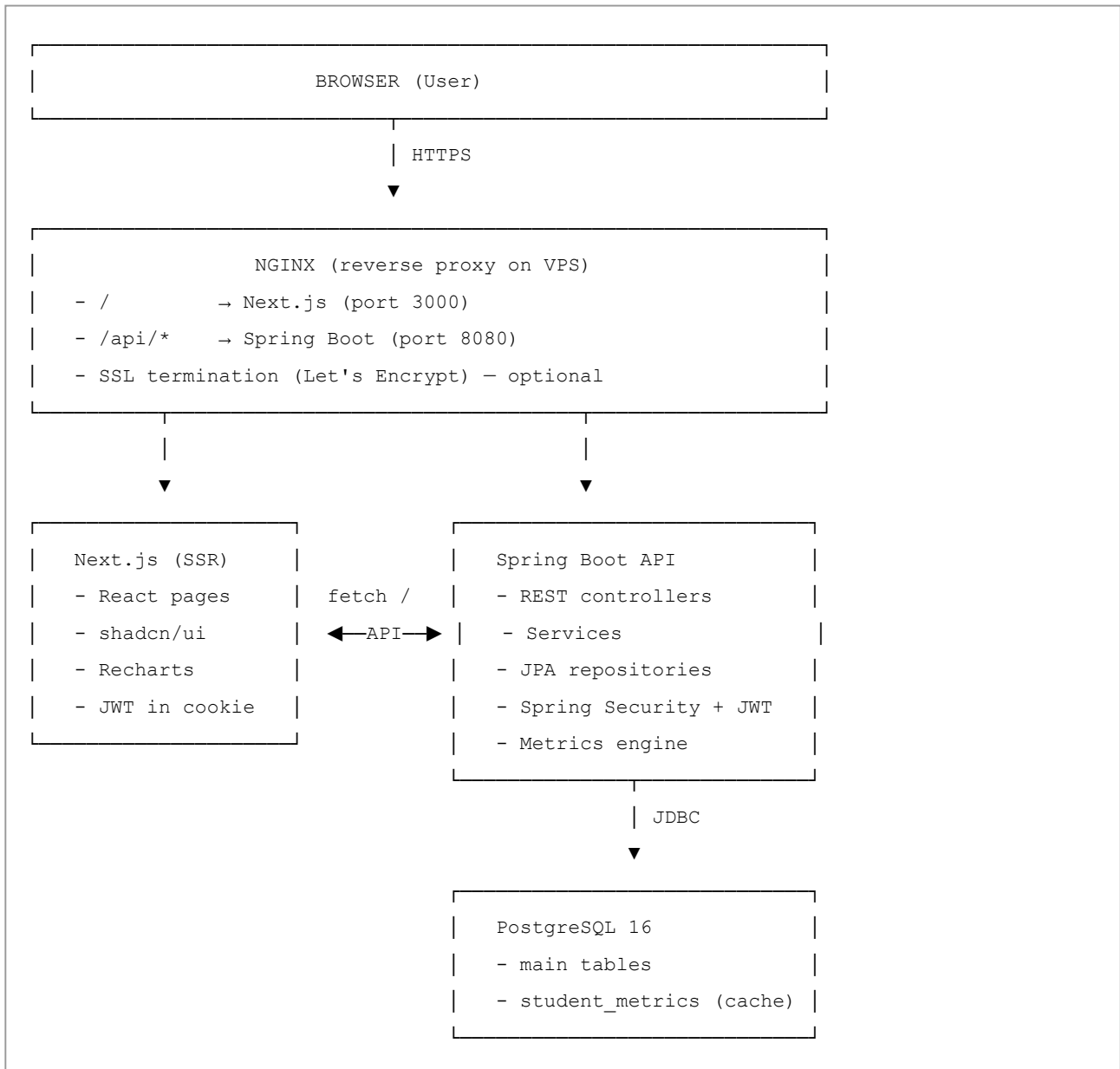
Critical because teachers use tablets in class. Touch targets ≥ 44 px, sticky save bar bottom, status buttons compact pills.

17.7 Phone (<640px)

Acceptance level: **works but not optimized**. Hackathon priority: login + dashboard must read correctly. Anything beyond that is a bonus.

18. Backend Architecture

18.1 High-level architecture



Everything in a single `docker-compose.yml` on one VPS. Three containers: frontend, backend, postgres. Optional fourth: nginx.

18.2 Architectural decisions

- **Monolith.** No microservices.
- **PostgreSQL.** No NoSQL.
- **No real-time.** No websockets. Page-refresh and short polling are sufficient.
- **No event-driven architecture.** Function calls within transactions.
- **Sync metrics recomputation.** No queues, no workers in MVP.
- **One optional cron** for snapshot creation.

18.3 Auth flow

1. User submits credentials → `POST /api/auth/login`
2. Backend verifies, generates JWT (HS256, 24h), returns in response + `httpOnly` cookie
3. Frontend stores token in `httpOnly` cookie (set via Next.js API route)
4. Subsequent requests include `Authorization: Bearer <token>` or cookie
5. Backend `JwtAuthenticationFilter` parses, populates `SecurityContext`
6. No refresh tokens in MVP. Expired = re-login.

18.4 API flow example (bulk attendance)

1. Frontend → `POST /api/attendance/bulk` (single request, all students)
2. `AttendanceController` → `AttendanceService.bulkUpsert(...)`
3. Inside one transaction:
 - Upsert records in attendance
 - For each affected `student_id`, call `MetricsService.recompute(studentId)`
 - Write to `audit_log` (minimal)
4. Response: updated entries + 200 OK

18.5 Dashboard loading flow

`GET /api/students/{id}/dashboard` — one endpoint, one response with everything: student info, current metrics, trend, recent grades/attendance, group comparison, growth areas.

Principle: one screen = one API call.

19. Frontend Architecture

19.1 Folder layout (Next.js 14+ App Router)

```

src/
├─ app/
│   ├─ layout.tsx
│   ├─ globals.css
│   ├─ page.tsx (redirect to login)
│   ├─ login/page.tsx
│   ├─ (dashboard)/
│   │   ├─ layout.tsx (sidebar + topbar)
│   │   ├─ admin/
│   │   │   ├─ page.tsx
│   │   │   ├─ students/{,[id]}/page.tsx
│   │   │   ├─ groups/
│   │   │   ├─ teachers/
│   │   │   └─ settings/
│   │   └─ teacher/
│   │       ├─ page.tsx (Today)
│   │       ├─ groups/{,[id]}/page.tsx
│   │       ├─ attendance/[lessonId]/page.tsx
│   │       └─ grades/[assignmentId]/page.tsx
│   └─ student/
│       ├─ page.tsx (Dashboard)
│       └─ profile/page.tsx
├─ api/auth/{login,logout}/route.ts
├─ components/
│   ├─ ui/ (shadcn)
│   ├─ layout/ (Sidebar, Topbar, PageContainer)
│   ├─ charts/ (ScoreRadarChart, TrendLineChart, ...)
│   ├─ metrics/ (ScoreCard, MetricBreakdown, AtRiskBadge)
│   ├─ attendance/ (BulkAttendanceForm, AttendanceRow)
│   ├─ students/
│   └─ common/ (DataTable, EmptyState, LoadingSkeleton)
├─ lib/
│   ├─ api/ (client.ts + per-domain API files)
│   ├─ auth/ (session.ts, middleware helpers)
│   ├─ types/ (matching backend DTOs)
│   └─ utils/
├─ hooks/ (useAuth, useStudents, useDashboard)
└─ middleware.ts (route protection)

```

19.2 Auth handling

- JWT in httpOnly cookie, set via Next.js API route proxy to backend `/api/auth/login`
- `middleware.ts` checks cookie on protected routes, redirects to `/login` if missing
- Server components: fetch with cookie forwarded

- Client components: fetch via `lib/api/client.ts` (credentials included)

19.3 API layer

Single fetch wrapper in `lib/api/client.ts`. Domain files (`students.ts`, `attendance.ts`, ...) are thin typed wrappers.

19.4 Data fetching strategy

- **SWR** for client-side fetching (lighter than React Query for this scope)
 - Server components only for initial page load
 - All interactive screens are client components with SWR
-

20. Database Structure

20.1 Tables (migration order)

V1 — Core: `users`, `teachers`, `students`, `groups`, `courses` **V2 — Domain:** `lessons`, `attendance`, `assignments`, `grades`, `behavior_records`, `activity_records` **V3 — Metrics:** `formula_config`, `student_metrics`, `metric_snapshots` **V4 — Audit:** `audit_log`

20.2 Key schema (concise)

```

users (id, email UNIQUE, password_hash, role CHECK IN ('ADMIN','TEACHER','STUDENT'),
      full_name, timestamps, deleted_at)

students (id, user_id FK UNIQUE, group_id FK, enrolled_at, timestamps, deleted_at)
teachers (id, user_id FK UNIQUE, department, timestamps, deleted_at)
groups   (id, name, course_id FK, start_date, end_date, timestamps, deleted_at)
courses (id, name, code UNIQUE, description, timestamps, deleted_at)

lessons (id, group_id FK, course_id FK, teacher_id FK, topic, scheduled_at, timestamps)

attendance (id, student_id FK, lesson_id FK, status CHECK IN (...),
           marked_by_user_id FK, marked_at, comment,
           UNIQUE(student_id, lesson_id))

assignments (id, course_id FK, name, type CHECK IN ('THEORY','PRACTICAL','PROJECT','EXAM'),
            max_value, weight, due_date, timestamps)

grades (id, student_id FK, assignment_id FK, value, graded_by_user_id FK, graded_at,
       comment, UNIQUE(student_id, assignment_id), CHECK(value >= 0))

behavior_records (id, student_id FK, teacher_id FK, period_start, period_end,
                 value CHECK BETWEEN 1 AND 5, comment, created_at,
                 UNIQUE(student_id, teacher_id, period_start, period_end))

activity_records (similar structure to behavior_records)

formula_config (id, version UNIQUE, weight_grades, weight_attendance, weight_practical,
               weight_behavior, weight_activity, weight_growth, weight_consistency,
               is_active, created_at)

student_metrics (id, student_id FK UNIQUE, composite_score, grades_norm, attendance_norm,
                 practical_norm, behavior_norm, activity_norm, growth_bonus,
                 consistency_bonus, formula_version, sample_size, computed_at)

metric_snapshots (id, student_id FK, snapshot_date, composite_score, all *_norm,
                  UNIQUE(student_id, snapshot_date))

audit_log (id, entity_type, entity_id, action, actor_user_id FK, payload JSONB, created_at)

```

20.3 Indexes

- All FK columns
- students.group_id WHERE deleted_at IS NULL
- lessons (group_id, scheduled_at)

- `attendance.student_id, attendance.lesson_id`
- `grades.student_id, grades.assignment_id`
- `behavior_records (student_id, period_end)`
- `activity_records (student_id, period_end)`
- `metric_snapshots (student_id, snapshot_date)`
- `audit_log (entity_type, entity_id)`

20.4 Integrity rules

- FK constraints everywhere with `ON DELETE RESTRICT` + soft delete on top-level entities
- CHECK constraints on enum-like fields and value ranges
- UNIQUE constraints to prevent duplicates: (student, lesson) for attendance, (student, assignment) for grades, etc.
- Transactions for all multi-row operations
- Soft delete on: users, students, teachers, groups, courses. NOT on attendance, grades, behavior_records, activity_records — these are immutable history.

20.5 Tooling

- Flyway for migrations (`V{n}__{description}.sql`)
- `hibernate.ddl-auto: validate` (never update)
- Seed data via dedicated repeatable migration or `CommandLineRunner` in dev profile

21. API Structure

All endpoints return JSON. Timestamps in ISO-8601 UTC. Errors: `{ "error": "...", "details": [...] }`.

21.1 Auth

- `POST /api/auth/login` → `{ token, user }`
- `POST /api/auth/logout` → `204`
- `GET /api/auth/me` → current user

21.2 Users (ADMIN)

- `POST /api/users` — create
- `GET /api/users?page=&size=&role=` — paged list

21.3 Students

- `GET /api/students?page=&size=&groupId=&search=` — paged
- `GET /api/students/{id}` — full
- `GET /api/students/{id}/dashboard` — composite dashboard payload (the most important endpoint)
- `POST /api/students` — ADMIN, create
- `PATCH /api/students/{id}` — partial update

- DELETE /api/students/{id} — soft delete

21.4 Groups, Courses, Teachers, Lessons

CRUD endpoints following the same pattern. Plus:

- GET /api/groups/{id}/students — students in group with current metrics
- GET /api/groups/{id}/analytics — group-level analytics
- GET /api/lessons/today — today's lessons for current teacher
- GET /api/lessons/{id}/attendance — attendance for lesson

21.5 Attendance

- POST /api/attendance/bulk — bulk upsert for a lesson (the high-frequency endpoint)
- GET /api/attendance/student/{id}?from=&to= — student attendance history

21.6 Grades

- POST /api/grades — single
- POST /api/grades/bulk — bulk for an assignment
- PATCH /api/grades/{id}, DELETE /api/grades/{id}
- POST /api/assignments, GET /api/assignments?courseId=

21.7 Behavior & Activity

- POST /api/behavior — single record
- POST /api/behavior/bulk — for a group, one period
- POST /api/activity — analogous

21.8 Metrics

- GET /api/students/{id}/metrics — current cached
- GET /api/students/{id}/metrics/trend?from=&to= — snapshots
- POST /api/metrics/recompute/{studentId} — ADMIN
- POST /api/metrics/recompute-all — ADMIN
- GET /api/metrics/formula — current active formula
- PUT /api/metrics/formula — ADMIN, updates weights + triggers recompute-all

21.9 Analytics

- GET /api/analytics/admin/dashboard — KPIs, top/weakest groups, teacher activity
 - GET /api/analytics/groups/{id} — group-level breakdown
 - GET /api/analytics/at-risk — at-risk students for current role's scope
-

22. Metrics Engine

22.1 Approach

- **Synchronous, in-transaction** recompute on data change
- One denormalized cache table (`student_metrics`)
- Weekly snapshots in `metric_snapshots` for trend and growth calculations
- No queues, no workers, no eventual consistency

22.2 Recompute triggers

- Attendance, grade, behavior, or activity insert/update → recompute affected student(s)
- Formula change → mark all stale, batch-recompute all students (paged, foreach)
- Dashboard open → if metrics are stale relative to last data change, lazy recompute

22.3 Compute method (pseudocode)

```
compute(studentId):  
    formula = formulaConfigRepo.findActive()  
    grades = gradeRepo.findByStudent(studentId)  
    attendance = attendanceRepo.findByStudent(studentId)  
    behavior = behaviorRepo.findByStudent(studentId)  
    activity = activityRepo.findByStudent(studentId)  
  
    gradesNorm = computeGradesNorm(grades)  
    attendanceNorm = computeAttendanceNorm(attendance)  
    practicalNorm = computeGradesNorm(filter(grades, PRACTICAL | PROJECT))  
    behaviorNorm = computeBehaviorNorm(behavior)  
    activityNorm = computeActivityNorm(activity)  
    growthBonus = computeGrowthBonus(studentId)  
    consistencyBonus = computeConsistencyBonus(studentId)  
  
    composite = sum(norm_i * weight_i) clamped to [0,100]  
  
    return result + sampleSize + formulaVersion
```

22.4 Snapshot job

Weekly cron writes one row per student into `metric_snapshots`. Used by trend chart and growth bonus calculation. For demo, seed includes 12 weeks of synthetic snapshots so trend charts look populated.

23. Scoring Formula

23.1 Formula

```
composite = w_g * grades_norm
           + w_a * attendance_norm
           + w_p * practical_norm
           + w_b * behavior_norm
           + w_ac * activity_norm
           + w_gr * growth_bonus
           + w_c * consistency_bonus
```

23.2 Default weights (sum = 1.0)

Dimension	Weight
Grades	0.25
Attendance	0.15
Practical	0.25
Behavior	0.10
Activity	0.10
Growth bonus	0.10
Consistency	0.05

All normalized values are 0–100. Composite is also 0–100.

23.3 Normalization

- **Grades:** weighted average of `value / max_value * 100` across all grades, weighted by `assignment.weight`. Empty → neutral default 50.
- **Attendance:** counted statuses (`PRESENT=1.0`, `LATE=0.7`, `ABSENT=0.0`). `EXCUSED` removed from denominator (no penalty). Result × 100.
- **Practical:** same formula as grades, restricted to assignments of type `PRACTICAL` or `PROJECT`.
- **Behavior:** average of teacher inputs (1–5 scale), mapped to 0–100 via $(\text{avg} - 1) / 4 * 100$.
- **Activity:** same as behavior.

23.4 Growth bonus

Compares last 4 weeks of snapshots to previous 4 weeks.

- $\Delta = \text{recentAvg} - \text{previousAvg}$
- `bonus = clamp(50 +  $\Delta \times 5$ , 0, 100)`
- Insufficient data → neutral 50

23.5 Consistency bonus

Standard deviation of last 8 weekly snapshots, mapped inversely to 0–100. Stable score → high consistency bonus. Volatile → lower.

- $\text{bonus} = \max(0, 100 - \text{stddev} \times 5)$
- Less than 3 snapshots → neutral 50

23.6 Fairness considerations

- **Insufficient data floor:** if sample size is too small (< 5 grades), the composite is hidden and "Insufficient data" is shown.
- **Excuse mechanism:** documented absences don't penalize attendance.
- **Per-group normalization:** future enhancement for teacher bias correction (z-score within group).
- **Transparency:** every score can be expanded to show full breakdown.
- **Formula versioning:** every metric records `formula_version`. When weights change, all metrics are recomputed.

24. Backend Modules

24.1 Package layout (Spring Boot)

```
com.edumetric/
├─ EduMetricApplication.java
├─ config/           (SecurityConfig, JwtConfig, CorsConfig, OpenApiConfig, WebConfig)
├─ security/         (JwtAuthenticationFilter, JwtTokenProvider, CustomUserDetailsService)
├─ common/           (dto, exception, audit, util)
├─ auth/             (AuthController, AuthService, dto)
├─ users/            (User entity, Role enum, controller, service, repository, dto)
├─ students/         (Student, controller, service, StudentDashboardService, dto)
├─ teachers/
├─ groups/
├─ courses/
├─ lessons/
├─ attendance/       (Attendance, AttendanceStatus enum, controller, service, dto)
├─ grades/           (Grade, Assignment, AssignmentType enum)
├─ behavior/         (BehaviorRecord, ActivityRecord, controllers, services)
├─ metrics/          (StudentMetrics, MetricSnapshot, FormulaConfig,
│                    MetricsEngine, MetricsService, controller, dto)
├─ analytics/        (AnalyticsController, AnalyticsService, dto)
└─ audit/            (AuditLog, AuditLogService)
```

24.2 Conventions

- Controllers return `ResponseEntity<ApiResponse<T>>`
- All write methods `@Transactional`, `read-only @Transactional(readOnly = true)`
- Authorization at method level: `@PreAuthorize("hasRole('TEACHER')")` etc.
- Never return entities from controllers — always map to DTO
- DTOs as Java records (Java 21)

- No MapStruct (overkill for hackathon) — simple static mappers
- Repositories: Spring Data JPA with `@Query` (JPQL) and native where necessary
- Always paginated list endpoints

24.3 Exception handling

`GlobalExceptionHandler` with `@RestControllerAdvice` handles `ResourceNotFoundException`, `BadRequestException`, `ForbiddenException`, `ConflictException`, `MethodArgumentNotValidException`, `AccessDeniedException`, and a catch-all `Exception`.

24.4 Security configuration

- Stateless session
 - CSRF disabled (JWT-based)
 - CORS configured for frontend origin
 - Public endpoints: `/api/auth/login`, `/api/auth/register`, `/api/health`, `/swagger-ui/**`
 - All others authenticated
 - BCrypt password encoder
 - `@EnableMethodSecurity` for `@PreAuthorize`
-

25. Frontend Pages

25.1 Page inventory

Page	Roles	Priority
<code>/login</code>	public	P0
<code>/student</code>	STUDENT	P0
<code>/student/profile</code>	STUDENT	P1
<code>/teacher</code>	TEACHER	P0
<code>/teacher/groups</code>	TEACHER	P1
<code>/teacher/groups/[id]</code>	TEACHER	P1
<code>/teacher/attendance/[id]</code>	TEACHER	P0
<code>/teacher/grades/[id]</code>	TEACHER	P1
<code>/admin</code>	ADMIN	P0
<code>/admin/students</code>	ADMIN	P0
<code>/admin/students/[id]</code>	ADMIN, TEACHER	P0
<code>/admin/groups</code>	ADMIN	P1
<code>/admin/teachers</code>	ADMIN	P1
<code>/admin/settings/formula</code>	ADMIN	P2
<code>/at-risk</code>	ADMIN, TEACHER	P1

25.2 Page composition pattern

Every page consists of:

- `PageHeader` (title, subtitle, optional actions)
- A series of `Card` components
- Each card holds either a chart, a table, or a metric block

This consistency makes the entire product feel cohesive and reduces frontend work.

26. Development Workflow

26.1 Team and roles

- 2 Java Backend Developers
- 1 Frontend Developer (web only)
- 1 Junior DevOps
- 1 PM

26.2 Backend work split

Backend Dev 1 — Core & Auth & CRUD:

- Project skeleton, Maven, Spring Boot
- Database connection, Flyway, V1 migration
- `config/`, `security/`, `common/`
- `auth/`, `users/`
- `students/`, `teachers/`, `groups/`, `courses/`
- Audit infrastructure
- Analytics endpoints (Day 4)

Backend Dev 2 — Domain & Metrics:

- V2 migration (`lessons/attendance/grades/behavior/activity`)
- `lessons/`, `attendance/`, `grades/`, `behavior/`
- **MetricsEngine**, **MetricsService** (the centerpiece)
- V3 migration (`formula_config`, `student_metrics`, `metric_snapshots`)
- `StudentDashboardService` (dashboard composition)

26.3 Frontend work split

Single developer, but a clear order:

- Day 1: skeleton, login, layout, API client, types
- Day 2: admin dashboard, list pages
- Day 3: **student dashboard** (the wow page), teacher today, attendance form
- Day 4: grades, group detail, at-risk view, polish
- Day 5: bug fixes, dark mode (optional), demo rehearsal

26.4 DevOps work split

- Day 1: VPS, Docker, GitHub repo, basic compose
- Day 2: Dockerfiles, full compose, nginx config
- Day 3: GitHub Actions, deployment automation
- Day 4: domain/SSL (optional), monitoring basics
- Day 5: **final deploy 6+ hours before demo**, smoke tests, standby

26.5 Critical dependencies

1. Backend Dev 1 → auth done (Day 1 EOD) → frontend can build protected routes
2. Backend Dev 1 → core entities (Day 1 EOD) → Dev 2 can reference in domain entities
3. Backend Dev 2 → attendance/grade endpoints (Day 2) → frontend builds teacher forms
4. Backend Dev 2 → metrics endpoint (Day 3 EOD) → frontend integrates dashboard
5. DevOps → deploy pipeline (Day 2–3) → daily deployments possible

27. Git Workflow

27.1 Branch model

Trunk-based with feature branches, merged via PR.

27.2 Branch naming

- `feat/` — new feature
- `fix/` — bug fix
- `chore/` — infrastructure, refactoring
- `docs/` — documentation

Example: `feat/student-dashboard-radar`

27.3 Commit messages (Conventional Commits, simplified)

```
feat: add bulk attendance endpoint
fix: correct growth bonus calculation for new students
chore: setup docker-compose for local dev
refactor: extract MetricsEngine to separate class
```

Subject ≤ 60 characters.

27.4 PR rules

- All changes to `main` via PR
- No direct push to `main`

- Minimum 1 reviewer (relaxed in final 24h with notification)
- Squash and merge
- Small PRs over large ones

27.5 Main branch protection

In GitHub Settings → Branches:

- Require PR before merging
 - Disable force push to main
 - Disable deletions
-

28. DevOps Structure

28.1 Deployment topology

- One VPS (Ubuntu 22.04 / 24.04)
- One `docker-compose.yml` with 3 services: `frontend`, `backend`, `postgres`
- Optional 4th: `nginx` (or host-level `nginx`)
- Optional SSL via Let's Encrypt + `certbot`

28.2 Backend container

- Multi-stage Dockerfile (Maven build → JRE 21 runtime)
- JAR runs as non-root
- Env vars: DB URL, JWT secret, profile
- Health endpoint: `/api/health`

28.3 Frontend container

- Next.js standalone output
- Node runtime
- Env vars: API URL

28.4 Postgres container

- Postgres 16 official image
- Named volume for data persistence
- Daily `pg_dump` cron, written to mounted volume

28.5 Nginx config

- Reverse proxy: `/` → `frontend`, `/api/*` → `backend`
- Gzip compression
- Optional SSL termination

28.6 CI/CD (GitHub Actions)

- On push to feature branch: build and test backend (Maven), lint frontend
- On merge to main: build Docker images, push to registry or pull on VPS
- Manual deploy script as backup

28.7 Monitoring (minimal)

- Docker logs to file with rotation
- Simple cron + curl uptime check, optionally telegram alert
- Nothing more elaborate for MVP

28.8 Safety nets

- Daily VPS-level backup (DigitalOcean/Hetzner snapshot, or manual pg_dump archived)
 - Local backup laptop with running build ready for demo emergencies
 - Demo video recording as fallback
-

29. Demo Strategy

29.1 Demo arc (8–10 minutes)

0:00–0:45 — Problem "GPA is one number. A person is not one number." Open with a transcript example. Pain: students don't know where to grow, teachers miss early warning signs, admins decide blindly.

0:45–1:30 — Solution in one sentence "EduMetric measures student growth across 7 dimensions, with early warning for at-risk students." Open straight to a student dashboard.

1:30–4:00 — Live demo (the core)

1. **Student view** (60s): Radar chart, trend line, growth areas. The first wow.
2. **Teacher view** (45s): Today's lessons → bulk attendance form → keyboard-driven input → save. Second wow: speed.
3. **Live recompute** (30s): Switch back to student tab → refresh → score reflects the new attendance. Synchronous architecture pays off here.
4. **Admin view** (45s): KPI cards, at-risk list with reasons. Third wow: actionable insight.
5. **Transparency modal** (30s): Click composite score → full formula breakdown shown. Fourth wow: explainability.

4:00–5:30 — Technical architecture (for technical jury) Stack overview. Emphasize deliberate simplicity: sync metrics recompute, monolith, no event sourcing. Frame as a strength, not a limitation.

5:30–7:00 — Market and roadmap Target audience (IT academies, university programs). Why now (skills-based hiring shift). Roadmap: optional ML, mobile app, integrations.

7:00–8:00 — Q&A buffer

29.2 Pre-demo checklist

- All seed accounts working, credentials tested
- 2–3 "good demo students" memorized for storytelling
- Three browser profiles pre-logged-in (admin, teacher, student)
- Backup video recorded
- Local laptop with running build standby
- Network: tested at venue or tethered backup

29.3 What not to do during demo

- Don't create accounts live on stage
 - Don't refresh accounts to "reset" demo data — use prepared accounts
 - Don't claim AI / ML capabilities you don't have
 - Don't show debug screens, raw API responses, or admin internals not designed for show
-

30. WOW-Effect Elements

30.1 High ROI (do first)

1. **Count-up animation** on composite score (0 → 78.4 over 800ms). Makes the dashboard feel "live."
2. **Animated radar chart** on load (Recharts default, just enable).
3. **Live recompute demo trick**: SWR polling every 5–10s makes another tab update without websockets. Powerful side-by-side.
4. **Glowing pulsing dot** on the last trend point. Telegraphs "current state."
5. **At-risk pulse indicator** on student avatars in lists.

30.2 Medium ROI

6. **Smart alert cards** ("Your practical work improved 12% this month"). Hardcoded logic, but reads like AI insight.
7. **Transparency modal**: clicking a composite score shows the full formula with the math line-by-line. **The strongest single feature for jury credibility.**
8. **Dark mode toggle** (only if time allows).

30.3 Avoid

- 3D, particles, Lottie animations
- Sound effects
- Confetti (consider one minimal use, but skip if any doubt about taste)

30.4 Premium polish tricks

- `tabular-nums` everywhere
- Subtle gradients under line charts
- Skeleton perfectly mirroring final layout
- Smooth chart transitions on tab switches

- Sticky table headers
 - Numeric badges on sidebar items ("12 pending grades")
 - "Last updated 2 mins ago" labels
-

31. Biggest Risks

Risk 1 — Seed data looks like seed data

Mitigation: Realistic names (Uzbek/Russian/English), realistic distributions, hand-crafted "demo students" with narrative arcs. PM owns this on Days 3–4.

Risk 2 — Junior DevOps fails on final deploy

Mitigation: Deploy to prod on Day 2, not Day 5. Daily working version on VPS. Backend dev with SSH backup access. Local laptop fallback.

Risk 3 — Metrics formula breaks on edge cases

Mitigation: Backend Dev 2 sanity-checks seed data after engine is built. Default values for empty data. Explicit clamping to $[0, 100]$. Unit tests on the engine (minimum 10 cases).

Risk 4 — Frontend dev overloaded

Mitigation: Strict page priority list. Cut from the bottom up: dark mode → heatmap → growth deep dive → audit log. Never cut dashboard or attendance form.

Risk 5 — Backend over-engineering

Mitigation: PM/tech lead vetoes any "let's do it the right way" proposals during the 5 days. Principle: if it can be a function call, it is a function call.

Risk 6 — API contracts diverge between frontend and backend

Mitigation: Decide JSON naming convention on Day 1 morning (camelCase). Swagger UI live from Day 1. Single Notion/Google Doc with canonical examples.

Risk 7 — Demo accounts not working at demo time

Mitigation: Seed credentials hardcoded in migration. Three browser profiles always pre-logged-in. No live login during demo.

Risk 8 — Git conflicts on Day 3–4

Mitigation: Minimize shared files. `application.yml` owned by Dev 1. Daily rebase on main before starting work.

Risk 9 — Performance issues on realistic data

Mitigation: Denormalized `student_metrics` table. Indexes per plan. `EXPLAIN ANALYZE` on slow queries during Day 4 polish.

Risk 10 — Jury asks a curveball question

Mitigation: PM rehearses answers to:

- *Why not ML?* → "Transparent decisions on student outcomes; black-box predictions are an ethical issue."
 - *How does it scale?* → "Vertical to ~50k students per instance; partitioned by `school_id` beyond that; multi-tenant on roadmap."
 - *How is this different from Moodle?* → "Moodle is content delivery; we're evaluation analytics. Complementary, not competing."
-

32. Final Product Vision

EduMetric CRM is, at its core, a single bet: that **education software should stop reducing students to a number and instead show how they are growing.**

This bet is expressed in every layer of the product:

- The **formula** is transparent because trust matters more than sophistication.
- The **architecture** is simple because debuggable beats clever when humans depend on the result.
- The **UI** is minimal because educational software has earned a reputation for being ugly, and the first wow is just looking modern.
- The **growth dynamics** are first-class because static numbers describe yesterday, not tomorrow.
- The **teacher input UX** is fast because the system is only as good as the data going in.

For the **hackathon**, the product wins by being the most polished, most coherent, most credible solution in the room. Not the most ambitious — the most finished.

For the **future**, EduMetric has a clear runway: real schools as customers, AI predictions as a credible roadmap item, mobile and integrations as growth vectors, and a defensible ethical position that distinguishes it from algorithmic-judgment systems.

The team is small, the timeline is tight, the scope is deliberately narrow. That is exactly why this product can ship — and that is why it can be beautiful.

End of master document.